

Repository Notes: EGMN (Watch Time Prediction)

1 Repository overview (Q&A)

1.1 Q: What is this repository?

A: This is a PyTorch research codebase for **video watch-time prediction**, implementing the EGMN paper and several baselines, with preprocessing and experiments primarily for the public KuaiRec dataset.

1.2 Q: What are the main folders and entrypoints?

A: Models live in `model/`, dataset loading is in `dataloader/`, KuaiRec preprocessing is in `dataset/kuaiRec/`, runnable experiments are in `run_*.py`, and shared metrics/utilities are in `utils.py`.

1.3 Q: What metrics are reported by the run scripts?

A: The run scripts evaluate on `test` and report **MAE** (absolute error), **XAUC** (ordering agreement after sorting by predicted score), and **KL** divergence between binned empirical distributions.

1.4 Q: How do you run EGMN and baselines (example commands)?

A:

```
# Train + evaluate the proposed EGMN model on KuaiRec
python run_egmn.py --dataset_name kuaiRec --device cuda:0

# Baselines (examples)
python run_cread.py --dataset_name kuaiRec
python run_d2q.py --dataset_name kuaiRec
```

1.5 Q: What does the dataloader yield to the model?

A:

```
# Pseudocode reflecting dataloader/kuaiRec.py
for features, label in dataloaders["train"]:
    # features: dict[str, Tensor], label: normalized play_time
    y_hat = model.predict(features)
```

1.6 Q: What do the evaluation helper functions look like (schematically)?

A:

```
# utils.py (simplified)
def eval_mae(labels, scores):
    return np.mean(np.abs(labels - scores))

def eval_xauc(labels, preds):
    # ranks by prediction, then measures label ordering agreement
    ...

def eval_kl(samples_p, samples_q, bins=100, epsilon=1e-10):
    # binned empirical KL divergence
    ...
```

2 EGMN model (Q&A)

2.1 Q: What is EGMN in this repository?

A: EGMN (Exponential–Gaussian Mixture Network), implemented in `model/egmn.py`, predicts watch time by modeling a **mixture distribution** that captures short “quick-skip” behavior and longer watch-time behavior.

2.2 Q: What distribution family does EGMN use?

A: EGMN uses a mixture consisting of $1 \times$ Exponential (short peak near 0) plus $K \times$ Gaussian components for longer watches, implemented as Normals truncated below at 0, i.e. $p(y) = \pi_0 \text{Exp}(y; \lambda) + \sum_{k=1}^K \pi_k \text{TN}(y; \mu_k, \sigma_k, \text{lower} = 0)$ for $y \geq 0$.

2.3 Q: What does the architecture look like end-to-end?

A: The model encodes sparse/sequence/continuous features into a shared representation, feeds it through a shared MLP trunk, and uses separate heads to produce π logits, λ , μ , and σ , which parameterize the mixture used for training and prediction.

2.4 Q: What parameters (heads) does EGMN output in code?

A:

```
# model/egmn.py (illustrative excerpt)
self.lambda_layer = nn.Sequential(
    nn.Linear(hidden_dim, 1),
    nn.Softplus(beta=0.5),
)

comp_num = 10 # number of Gaussian components
self.mixture_logits = nn.Linear(hidden_dim, comp_num + 1) # +1 for Exponential
self.gauss_mu = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
self.gauss_sigma = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
```

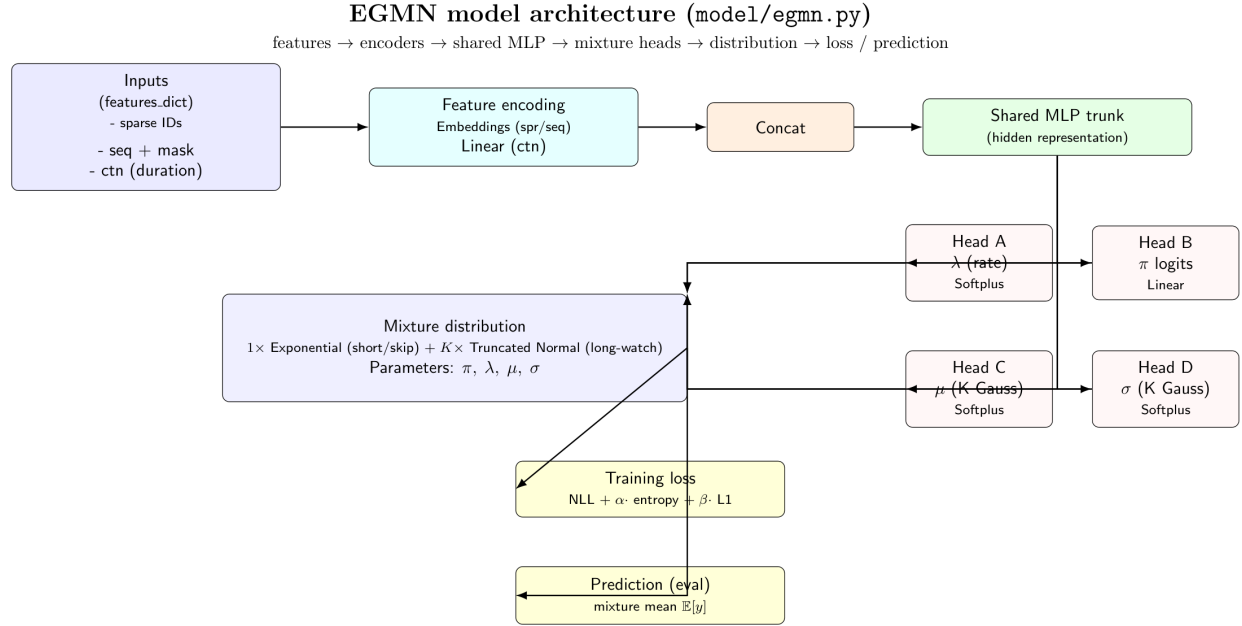


Figure 1: EGMN architecture overview: feature encoders + shared MLP + mixture-parameter heads, producing a distribution and a mean prediction.

2.5 Q: What loss does EGMN train with in the run script?

A: Training minimizes a mixture negative log-likelihood plus regularizers: $\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}}$, where $\mathcal{L}_{\text{reg}}$ is an L1 reconstruction between label and mixture-mean prediction and $\mathcal{L}_{\text{entropy}}$ is a mixture-weight entropy term.

2.6 Q: What scalar output is used for evaluation?

A: The evaluation uses a point estimate equal to the **mixture mean** (including the exponential mean $1/\lambda$), producing one scalar prediction per example.

2.7 Q: How is the mixture-mean prediction computed (schematically)?

A:

```
# model/egmn.py (illustrative excerpt)
pi, lambda_, mu, sigma = self.forward(features)
pi = torch.softmax(pi, dim=1)
component_means = torch.concat([1 / lambda_, mu], dim=1)
pred = torch.sum(pi * component_means, dim=1)
```

3 Retrieval vs. ranking vs. outputs (Q&A)

3.1 Q: Is this a retrieval model?

A: No: the provided scripts score already-observed (user, video) interaction records from a dataset loader and do not implement large-scale candidate generation (ANN/top- K retrieval) over a corpus.

3.2 Q: Is this a ranking model?

A: Not in the training-objective sense: the training loops use pointwise losses (likelihood/regression-style) per example rather than pairwise/listwise ranking losses over candidate sets.

3.3 Q: What is the final output used for evaluation?

A: Each model produces a scalar watch-time prediction per example for evaluation (e.g. EGMN uses the mixture mean; other baselines output scalars or reconstruct scalars from intermediate representations).

3.4 Q: What does EGMN output internally?

A: EGMN outputs mixture-parameter tensors π logits, λ , and (μ, σ) for multiple Gaussian components, which define the mixture distribution and its mean prediction.

3.5 Q: What does the EGMN evaluation loop look like (schematically)?

A:

```
with torch.no_grad():
    for features, label in dataloaders["test"]:
        y = model.predict(features) # scalar per example
```

3.6 Q: What are typical output shapes for some baselines?

A:

```
# Wide&Deep: scalar
p = wide_and_deep(features) # [batch]

# CREAD: per-bucket/head probabilities
bucket_probs = cread(features) # [batch, M]

# D2Q: scalar quantile (later mapped back to value)
q = d2q(features) # [batch]
```

3.7 Q: If training is pointwise, why report XAUC?

A: XAUC is a ranking-style evaluation metric computed by sorting examples by predicted score and measuring label-order agreement, but it does not change the fact that the provided training scripts optimize pointwise objectives.

4 Optimization / gradient descent (Q&A)

4.1 Q: What does gradient descent update in these models?

A: Gradient-based optimizers update every learnable tensor returned by `model.parameters()` (embeddings, linear layers, MLP weights, and mixture-head weights), because those parameters determine the predicted distribution/score and therefore the loss.

4.2 Q: What objective is being minimized for EGMN in training?

A: The training script minimizes $\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}}$, i.e. mixture negative log-likelihood plus an entropy term and an L1 reconstruction term weighted by α, β .

4.3 Q: What does one optimizer step look like in code?

A:

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=args.lr,
                                weight_decay=args.weight_decay)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

4.4 Q: How do gradients flow to EGMN's mixture outputs?

A: The outputs $\pi, \lambda, \mu, \sigma$ are not free parameters but functions of the network weights, so back-propagation differentiates the loss through these outputs into the head layers and shared trunk that produced them.

4.5 Q: Which things are not updated by gradient descent?

A: Inputs (feature/label tensors), preprocessing artifacts (e.g. bucket boundaries and quantile tables), and computations inside `torch.no_grad()` are not updated because they are not learnable parameters in the optimizer.

5 Workflow (Q&A)

5.1 Q: What preprocessing is required for KuaiRec before training?

A: You must run `dataset/kuaiREC/kuaiREC_process.py` to merge KuaiRec raw CSVs, build vocabularies for sparse/sequence features, create the feature schema (`description`), and write a preprocessed pickle used by the dataloader.

5.2 Q: What files does preprocessing produce that training depends on?

A: Preprocessing produces `kuaiREC_data.pkl` (train/test splits + schema) and also writes duration buckets and per-bucket play-time quantiles used by the D2Q baseline.

5.3 Q: What does the D2Q preprocessing artifact generation look like (schematically)?

A:

```
n_bins = 50
df["duration_bucket"], bins = pd.qcut(df["duration"], q=n_bins, labels=False,
                                     retbins=True, duplicates="drop")
desc.append(("duration_bucket", len(bins) - 1, "spr"))

quantile_num = 100
quantiles = np.linspace(0, 1, quantile_num + 1)
```

```

quantile_df = (df_train.groupby("duration_bucket")["play_time"]
               .quantile(quantiles).reset_index())
quantile_df.pivot(index="duration_bucket", columns="quantile",
                  values="play_time").to_csv(
    "d2q_duration_bucket_playtime_quantiles.csv"
)

```

5.4 Q: How do you run training/evaluation for a method?

A: Run the corresponding entrypoint (e.g. `python run_egmn.py --dataset_name kuairc`), which loads `./dataset/kuairc/kuairc_data.pkl` via `dataloader/kuairc.py`, trains for several epochs, evaluates on test, and prints MAE/XAUC/KL.

5.5 Q: What does the EGMN training loop structure look like (schematically)?

A:

```

for epoch in range(1, args.epoch + 1):
    for features, label in dataloaders["train"]:
        pi, lambda_, mu, sigma = model(features)
        nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                                   features["duration"].view(-1, 1))
        loss = nll + args.alpha * ent + args.beta * reg
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

5.6 Q: What dependencies does this repo assume?

A: The README indicates Python 3.x with PyTorch (tested around 2.1.0) and common scientific packages such as NumPy, pandas, and scikit-learn.